

Utilities (MidiNoteUtility / MidiMessageBuilder)

This page describes the runtime utilities that assist with MIDI 1.0 send/receive and related helpers.

Namespace: `jp.kshoji.unity.midi.util`

Location:

- `Assets/MIDI/Scripts/Utilities/MusicTheory/MidiNoteUtility.cs`
- `Assets/MIDI/Scripts/Utilities/Messaging/MidiMessageBuilder.cs`
- `Assets/MIDI/Scripts/Utilities/Messaging/PitchBendUtility.cs`
- `Assets/MIDI/Scripts/Utilities/Messaging/MidiControlSmoothing.cs`
- `Assets/MIDI/Scripts/Utilities/Messaging/AutomationInterpolation.cs`
- `Assets/MIDI/Scripts/Utilities/Smf/` — `TempoMapExtractor`, `MeasureTimeUtility`, `TupletUtility`, `SwingUtility`, `MidiMetaMessageFactory` (details in [SMF Tools](#))
- `Assets/MIDI/Scripts/Editor/UI/PianoKeyboardElement.cs` (`MidiKeyboardLogic`)

MidiNoteUtility

A static utility that provides conversion between MIDI note numbers and note names, octave calculation, semitone transposition, and more. It is a pure C# implementation with no dependency on Unity or the MIDI plugin.

Key constants

Constant	Value	Description
<code>MiddleC</code>	60	C4 in General MIDI
<code>MinNote</code>	0	Minimum note number
<code>MaxNote</code>	127	Maximum note number

API overview

Method	Description	Example
<code>ToNoteName(noteNumber, useSharps)</code>	Convert a note number to a note name	<code>60</code> → <code>"C4"</code>
<code>TryParseNoteName(name, out noteNumber)</code>	Convert a note name to a note number	<code>"C#4"</code> → <code>61</code>
<code>GetOctave(noteNumber)</code>	Get the octave number	<code>60</code> → <code>4</code>
<code>GetPitchClass(noteNumber)</code>	Get the pitch class (0–11)	<code>64</code> → <code>4</code> (E)
<code>Transpose(noteNumber, semitones)</code>	Transpose by semitones (clamped to 0–127)	<code>(60, 12)</code> → <code>72</code>
<code>FromPitchClassAndOctave(pitchClass, octave)</code>	Compute a note number from pitch class and octave	<code>(0, 4)</code> → <code>60</code>
<code>FormatNoteWithNumber(noteNumber)</code>	Show the note name together with its number	<code>60</code> → <code>"C4 (60)"</code>

Usage example

```
using jp.kshoji.unity.midi.util;

// Note number → note name
Debug.Log(MidiNoteUtility.ToNoteName(60));           // "C4"
Debug.Log(MidiNoteUtility.ToNoteName(61, useSharps: false)); // "Db4"

// Note name → note number
if (MidiNoteUtility.TryParseNoteName("C#4", out var note))
    Debug.Log(note); // 61

// Transpose
var transposed = MidiNoteUtility.Transpose(60, 7); // G4 (67)
```

Octave convention

This utility uses the standard MIDI octave notation.

$$\text{octave} = (\text{noteNumber} / 12) - 1$$

Examples: note number 60 → C4, 0 → C-1, 127 → G9

MidiMessageBuilder (MidiSend)

Wraps the send API of `MidiManager` in a fluent interface, providing chainable MIDI message sending. Internally it calls `MidiManager.Instance.SendMidi*`.

Entry points

Method	Description
<code>MidiSend.To(deviceId)</code>	Get a builder targeting the specified device
<code>MidiSend.ToFirstOutput()</code>	Get a builder targeting the first output device

If `deviceId` is empty, an `ArgumentException` is thrown; if no output device exists, an `InvalidOperationException` is thrown.

Builder methods

Method	Description
<code>Group(int)</code>	Set the MIDI 2.0 group (0–15, default 0)
<code>Channel(int)</code>	Set the MIDI channel (0–15, default 0)
<code>NoteOn(note, velocity)</code>	Send a Note On (velocity defaults to 127)
<code>NoteOff(note, velocity)</code>	Send a Note Off (velocity defaults to 0)
<code>ControlChange(controller, value)</code>	Send a Control Change
<code>ProgramChange(program)</code>	Send a Program Change
<code>PitchWheel(amount)</code>	Send pitch bend (14-bit, 0–16383; center 8192)
<code>PitchWheelNormalized(value)</code>	Send pitch bend from a normalized float (via <code>PitchBendUtility</code>)
<code>SystemExclusive(byte[])</code>	Send SysEx
<code>AllNotesOff()</code> / <code>AllNotesOff(channel)</code>	Send All Notes Off (CC 123) on all or one channel

Each method returns `this`, so calls can be chained.

Usage example

```
using jp.kshoji.unity.midi.util;

// Single send
MidiSend.To(deviceId).Channel(0).NoteOn(60, 100);

// Send a chord in sequence
MidiSend.To(deviceId).Channel(0)
    .NoteOn(60, 127)
    .NoteOn(64, 127)
    .NoteOn(67, 127);

// Send a CC to the first output device
MidiSend.ToFirstOutput().Channel(0).ControlChange(1, 64);
```

Relationship with MidiManager

`MidiSend` is a thin wrapper over `MidiManager`. It behaves the same as the existing `SendMidiNoteOn` and similar methods. In the Unity editor, sent messages appear as OUT in the [MIDI Monitor](#).

For the detailed send API, see also "Sending MIDI 1.0 messages" in [MIDI 1.0 Runtime \(MidiManager\)](#).

PitchBendUtility

14-bit pitch-bend helpers (center = 8192). Pure C#; no Unity dependency.

Location: `Assets/MIDI/Scripts/Utilities/Messaging/PitchBendUtility.cs`

Member	Description
<code>Min</code> / <code>Max</code> / <code>Center</code>	<code>0</code> / <code>16383</code> / <code>8192</code>
<code>Clamp(amount)</code>	Clamp to 0–16383
<code>FromNormalized(normalized, zeroToOne)</code>	Map float → 14-bit (<code>zeroToOne</code> : 0–1, or bipolar $-1\dots+1$)
<code>ToNormalized(amount)</code>	Map 14-bit → 0–1 (center ≈ 0.5)
<code>Split</code> / <code>Combine</code>	LSB/MSB (7+7 bit) pack/unpack

Semitone-range conversion is not included yet (see Future features in [index](#)).

```
var amount = PitchBendUtility.FromNormalized(0.75f);
PitchBendUtility.Split(amount, out var lsb, out var msb);
MidiSend.To(deviceId).Channel(0).PitchWheel(amount);
```

MidiControlSmoothing

Shared EMA (exponential moving average) helpers used by `MidiCcSmoother` and sequencer output stages.

Location: `Assets/MIDI/Scripts/Utilities/Messaging/MidiControlSmoothing.cs`

Method	Description
<code>ApplyEma(previous, raw, smoothFactor)</code>	EMA; <code>smoothFactor</code> 0.01–1.0 (smaller = smoother)
<code>ApplyEmaToMidi7(...)</code>	EMA then round/clamp to 0–127
<code>ApplyEmaToMidi14(...)</code>	EMA then round/clamp to 0–16383 (pitch bend)

AutomationInterpolation

Evaluates ordered automation breakpoints (normalized 0–1 values) at a tick.

Location: `Assets/MIDI/Scripts/Utilities/Messaging/AutomationInterpolation.cs`

API	Description
<code>CurveKind.Linear</code> / <code>Smooth</code>	Linear lerp or smoothstep between points
<code>Evaluate(points, tick, curve)</code>	Value at tick; before first / after last clamps to endpoint; empty → 0

MidiKeyboardLogic

A keyboard-layout utility shared between the editor's virtual MIDI controller keyboard and the `MidiKeyboardElement` on the runtime UI surface.

Location: `Assets/MIDI/Scripts/Editor/UI/PianoKeyboardElement.cs` (`MidiKeyboardLogic`)

API	Description
<code>IsBlackKey(note)</code>	Determine whether the note is a black key
<code>CountWhiteKeysBefore(note)</code>	Number of white keys before the specified note
<code>CountWhiteKeys(start, end)</code>	Number of white keys within a range
<code>DefaultStartNote</code> / <code>DefaultEndNote</code>	Default 2-octave range (C3–B4)

For details on the runtime UI, see [Genre Kits](#).

TempoMapExtractor

A static utility that extracts tempo changes and time signatures from an SMF (`Sequence`) as a time series. It can be used for `SmfPlayer` playback-time calculation and as a foundation for generating scores and timelines.

Location: `Assets/MIDI/Scripts/Utilities/Smf/TempoMapExtractor.cs`

API

Method	Description
<code>Extract(Sequence)</code>	Build a tempo map from a <code>Sequence</code>
<code>Extract(byte[] smfData)</code>	Extract from SMF binary
<code>Extract(string filePath)</code>	Extract from a <code>.mid</code> file

TempoMap

Member	Description
<code>TempoChanges</code>	Tempo changes (Tick / seconds / BPM)
<code>TimeSignatures</code>	Time signatures
<code>TotalDurationSeconds</code>	Total playback duration
<code>GetBpmAt(tick)</code>	BPM at the specified Tick
<code>TickToSeconds(tick)</code>	Tick → seconds
<code>SecondsToTick(seconds)</code>	seconds → Tick

For detailed usage examples, see [SMF Tools](#). The same folder also provides:

Utility	Role
<code>MeasureTimeUtility</code>	Bar length in ticks (<code>TicksPerBar</code> / <code>TicksPerBeat</code>), measure ↔ time
<code>TupletUtility</code>	Rational tuplet step ticks (e.g. 3:2); reversible index↔tick
<code>SwingUtility</code>	Playback-only swing delay on even offbeats
<code>MidiMetaMessageFactory</code>	Build/read Tempo, Time Signature, Track Name meta events

MidiScaleUtility

A static utility that determines whether a note belongs to a specified scale. It is also used by the **Is Note In Scale** node in Visual Scripting.

MidiScaleType

Value	Description
Major	Major scale
NaturalMinor	Natural minor scale
HarmonicMinor	Harmonic minor scale
MajorPentatonic	Major pentatonic
MinorPentatonic	Minor pentatonic
Blues	Blues
Dorian	Dorian
Mixolydian	Mixolydian
Chromatic	Chromatic

API

Method	Description
IsNoteInScale(noteNumber, rootNote, scaleType)	Whether the note is contained in the scale
GetIntervals(scaleType)	Array of semitone intervals for the scale

```
using jp.kshoji.unity.midi.util;

// D4 (62) is contained in the C major scale (root C4=60)
bool inScale = MidiScaleUtility.IsNoteInScale(62, 60, MidiScaleType.Major); // true
```

ScaleUtility / ChordRecognition

`ScaleUtility` extends `MidiScaleUtility` to determine scale membership for multiple notes at once.

Method	Description
<code>AllNotesInScale(notes, rootNote, scaleType)</code>	Whether all notes are in the scale
<code>AnyNoteOutsideScale(notes, rootNote, scaleType)</code>	Whether any note is outside the scale
<code>GetIntervals(scaleType)</code>	Alias for <code>MidiScaleUtility.GetIntervals</code>

`ChordRecognition.Recognize(activeNotes)` infers a chord name (`C` , `Cm7` , `Cmaj7` , etc.) from the set of held notes. It supports the main triad and seventh-chord patterns.

```
var notes = noteTracker.GetActiveNotes(0);
var chord = ChordRecognition.Recognize(notes);
var inScale = ScaleUtility.AllNotesInScale(notes, rootNote: 60, MidiScaleType.Major);
```

For detailed real-time detection, see `MidiChordDetector` in [Gameplay Components](#).

Related docs

- [MIDI 1.0 Runtime \(MidiManager\)](#)
- [SMF Tools \(SmfPlayer / MidiRecorder\)](#)
- [Editor Tools \(MIDI Monitor\)](#)
- [Getting Started](#)